

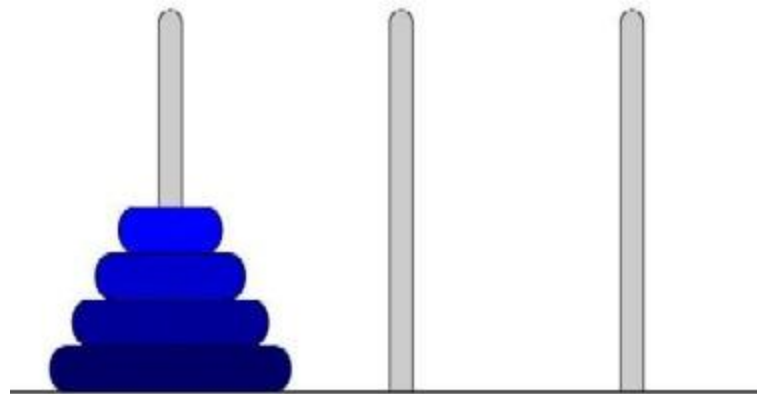
Data Structure

CSE207

Md. Manowarul Islam
Guest Lecturer
Dept. of CSE, SEU

Tower of Hanoi

- Tower of Hanoi is a mathematical puzzle invented by a French Mathematician Edouard Lucas in 1883.
- The game starts by having few discs stacked in increasing order of size. The number of discs can vary, but there are only three pegs.



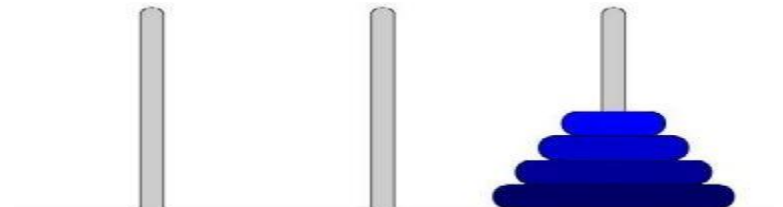
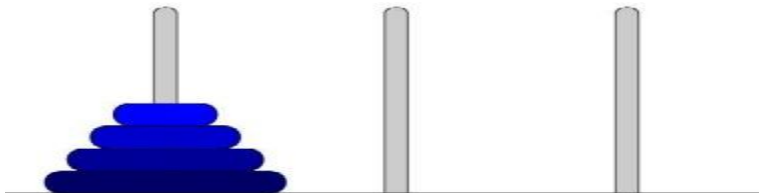
Tower of Hanoi

- Legend has it that there were three diamond needles set into the floor of the temple of Brahma in Hanoi.
- Stacked upon the leftmost needle were 64 golden disks, each a different size, stacked in concentric order
- The priests were to transfer the disks from the first needle to the third needle, using the second as necessary
- But they could *only move one disk at a time*, and could *never put a larger disk on top of a smaller one*.



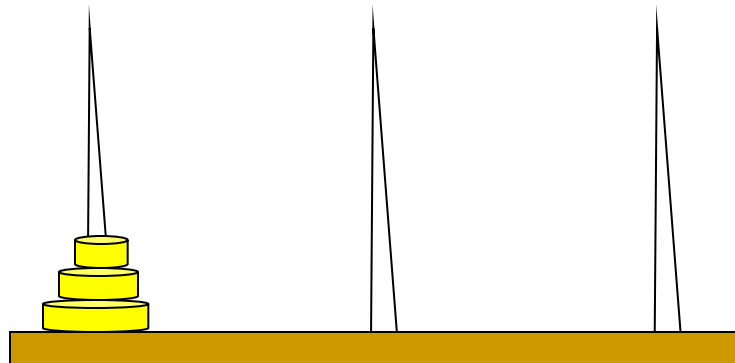
Tower of Hanoi

- When they completed this task,
- the world would end!!!!



Example

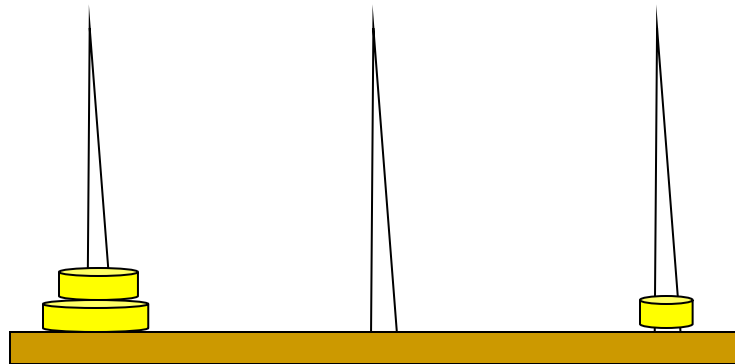
- For simplicity, suppose there were just 3 disks, and we'll refer to the three needles as A, B, and C
- Since we can only move one disk at a time, we move the top disk from A to C.



Example

Count# 1

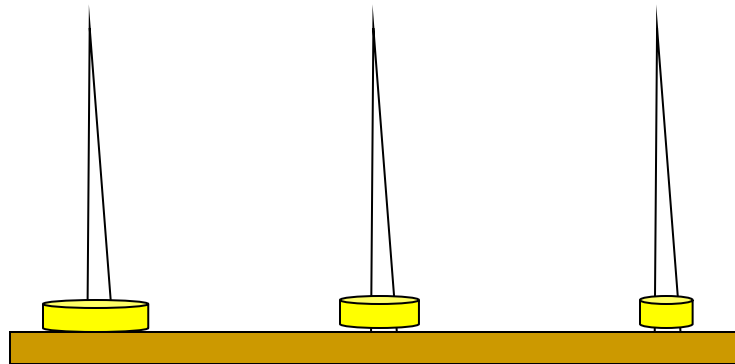
- Since we can only move one disk at a time, we move the top disk from A to C.



Example

Count# 2

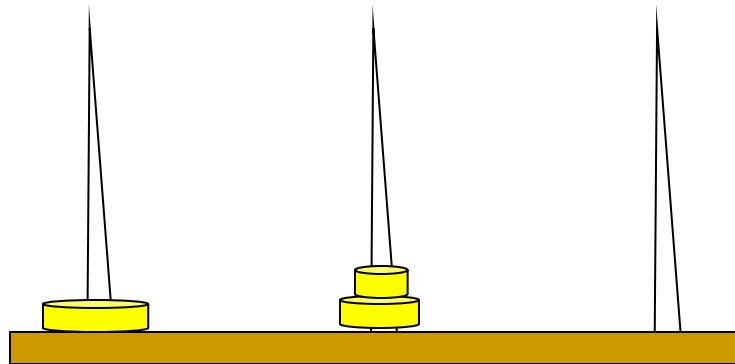
- we move the top disk from A to B.



Example

Count# 3

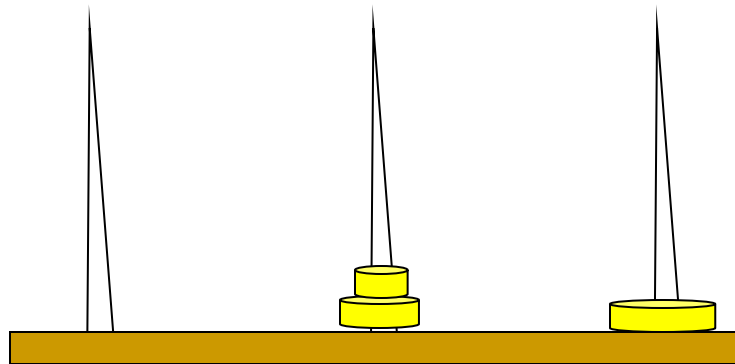
- we move the top disk from C to B.



Example

Count# 4

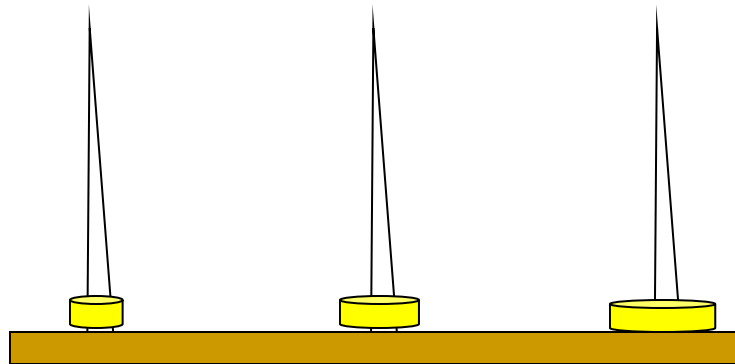
- we move the top disk from A to C.



Example

Count# 5

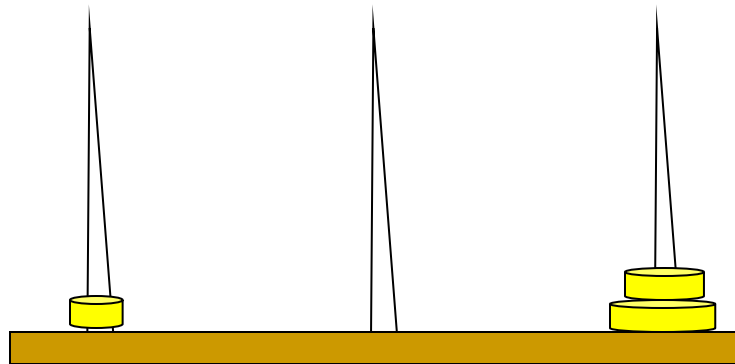
- we move the top disk from B to A.



Example

Count# 6

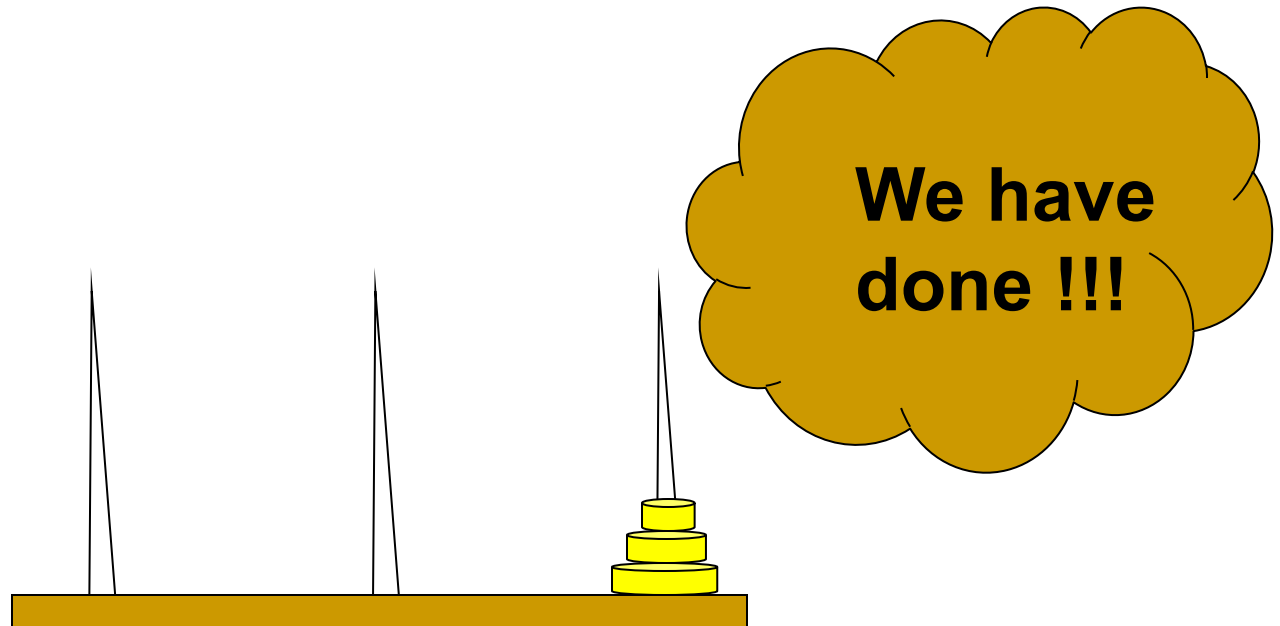
- we move the top disk from B to C.



Example

Count# 7

- we move the top disk from A to C.



How to solve the 4 disk



Tower of Hanoi

- Let q equal to the minimum number of moves necessary to complete the relocation. Examining the case when $d = 3$ we find $n = 7$.

$$d = 4 \quad n = 15$$

$$d = 5 \quad n = 31$$

$$d = 6 \quad n = 63$$

$$d = 7 \quad n = 127$$

$$d = 8 \quad n = 255$$

$$2^i - 1$$

$$2^{64} - 1 \text{ (a big number)}$$

How big

- Suppose that our computer and “super-printer” can generate and print 1,048,576 (2^{20}) instructions/second.
- How long will it take to print the priest's instructions?
- There are 2^{64} instructions to print.
 - Then it will take $2^{64}/2^{20} = 2^{44}$ *seconds* to print them.
- 1 minute == 60 seconds.
 - Let's take $64 = 2^6$ as an approximation of 60.
 - Then it will take $\cong 2^{44} / 2^6 = 2^{38}$ *minutes* to print them.

How big

- Hmm. 2^{38} minutes is hard to grasp. Let's keep going...
- 1 hour == 60 minutes.
 - Let's take $64 = 2^6$ as an approximation of 60.
 - Then it will take $\cong 2^{38} / 2^6 = 2^{32}$ *hours* to print them.
- 1 day == 24 hours.
 - Let's take $32 = 2^5$ as an approximation of 24.
 - Then it will take $\cong 2^{32} / 2^5 = 2^{27}$ *days* to print them.

How big

- Hmm. 2^{27} days is hard to grasp. Let's keep going...
- 1 year == 365 days.
 - Let's take $512 = 2^9$ as an approximation of 365.
 - Then it will take $\cong 2^{27} / 2^9 = 2^{18}$ *years* to print them.
- 1 century == 100 years.
 - Let's take $128 = 2^7$ as an approximation of 100.
 - Then it will take $\cong 2^{18} / 2^7 = 2^{11}$ *centuries* to print them.

How big

- Hmm. 2^{11} centuries is hard to grasp. Let's keep going...
- 1 millenium == 10 centuries.
 - Let's take $16 = 2^4$ as an approximation of 10.
 - Then it will take $\cong 2^{11} / 2^4 = 2^7 = 128$ *millenia* just to *print* the priest's instructions (assuming our computer doesn't crash, in which case we have to start all over again).

How fast can the priests actually *move* the disks?

I'll leave it to you to calculate the data.....

How big

- Parville's story the priests had to relocate a tower of 64 disks. Therefore:

$$q = 2^{64} - 1 = 18,446,744,073,709,551,615$$

- Supposing that the priests were proficient enough to make a move a second, the world would end in 584,942,417,355.1 years.

A Solution to the Towers of Hanoi Problem

The following algorithm is to find a recursive solution to the Towers of Hanoi problem.

Algorithm: Tower(N, Beg, Aux, End)

This algorithm gives a recursive solution to the Towers of Hanoi problem for N disks.

1. If $N = 0$ then Return.
2. If $N = 1$ then Write: $Beg \rightarrow End$ and Return.
3. [Move N-1 disks from Beg to Aux.]

Call Towers-of-Hanoi(N-1, Beg, End, Aux)

4. Write: $Beg \rightarrow End$.
5. [Move N-1 disks from Aux to End.]

Call Towers-of-Hanoi(N-1, Aux, Beg, End)

Example: Recursive solution to the Towers of Hanoi problem for N=3.

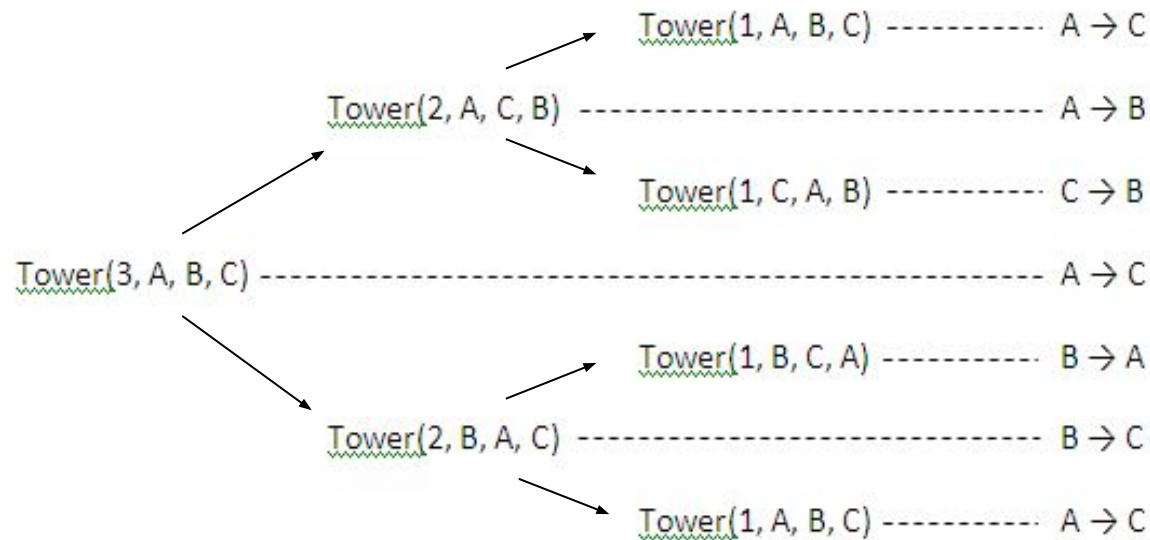
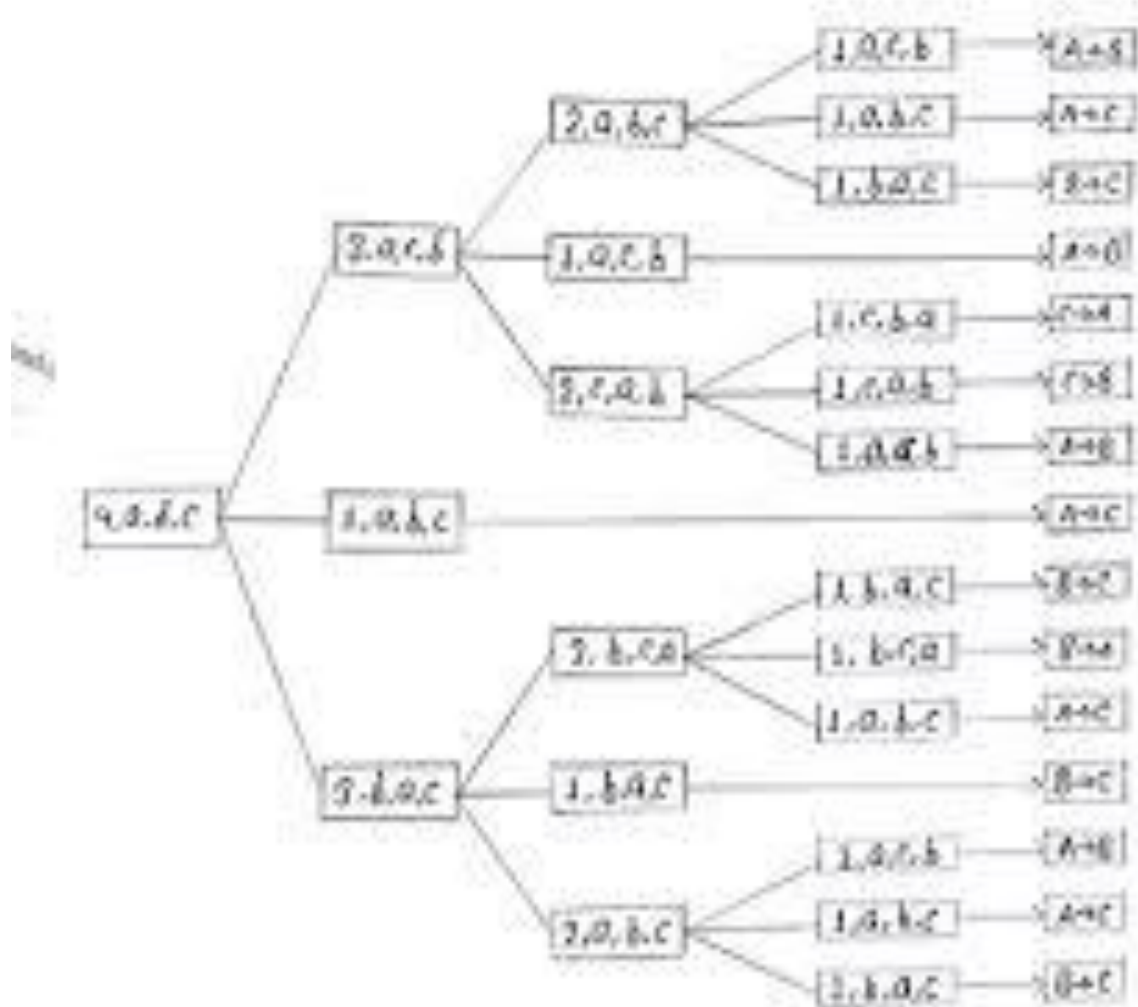


Figure: Recursive solution to the Towers of Hanoi problem for N=3.

Call Towers-of-Hanoi(N-1, Beg, End, Aux)
Beg → End.
Call Towers-of-Hanoi(N-1, Aux, Beg, End)

Assignment: Write a recursive solution to the Towers of Hanoi for N=5.





End